

SMART COMPLAINT MANAGEMENT SYSTEM

**AWS Serverless Microservices Using S3,Lambda, Api
Gateway and DynamoDB**

MAJOR PROJECT REPORT

Submitted by: S. Priyadharshini

Institution: Login360 | Batch: July | Timing: 11.30–01.00

Abstract

The Smart Complaint Management System is a serverless AWS web application that allows users to register complaints and track their status with a unique complaint ID. HTML, CSS and JavaScript are hosted in Amazon S3. The browser sends HTTPS requests to Amazon API Gateway REST API, which invokes AWS Lambda functions written in Python. Complaint data is stored in Amazon DynamoDB. This report describes the implemented architecture, service configuration, source code, example Lambda test events, deployment workflow and test results.

1. Introduction

The application provides a single browser interface for submitting and tracking complaints. AWS managed services separate static website hosting, API routing, application logic and database storage.

1.1 Project scope

- Register complaints using name, phone, category, location and description.
- Generate a complaint reference with CMP- prefix and set initial status to Pending.
- Track an existing complaint by its ID and display category and status.
- Provide a separate UpdateComplaint Lambda backend function for status changes; the browser interface exposes registration and tracking.

2. Problem statement and objectives

The application records complaint details in one database and enables reference-based status lookup without requiring users to contact an operator for every update.

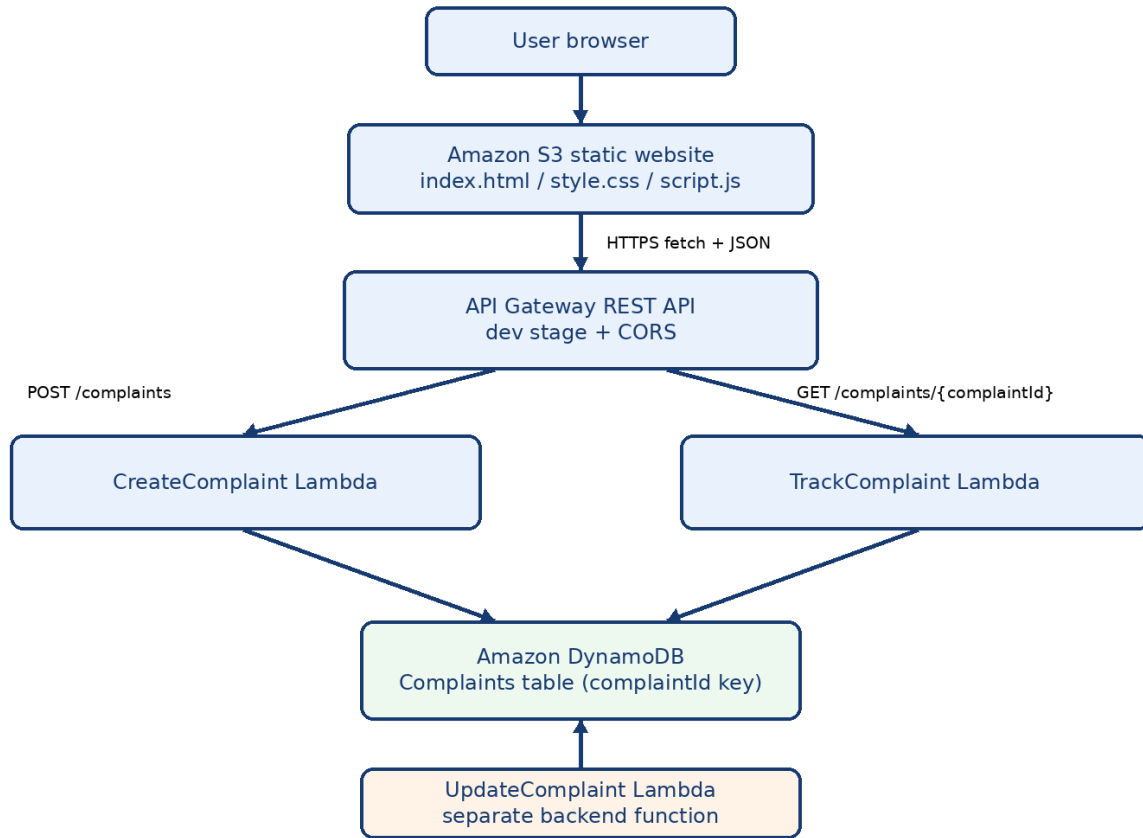
- Build a responsive static frontend.
- Expose REST endpoints for registration and tracking.
- Store and retrieve complaint records in DynamoDB.
- Use IAM roles to authorize Lambda access to the database.

3. Technologies and requirements

Component	Technology / purpose
Frontend	HTML, CSS, JavaScript
Website hosting	Amazon S3 static website hosting
API	Amazon API Gateway REST API, dev stage
Backend	AWS Lambda (Python and boto3)
Database	Amazon DynamoDB – Complaints table
Access control	AWS IAM execution roles and DynamoDB permissions

4. Complete system architecture

Figure 4.1: System architecture and data flow



The browser loads the S3-hosted frontend; API Gateway routes registration and tracking requests to their respective Lambda functions, which access the Complaints table.

Registration flow: Browser → S3-hosted frontend → API Gateway POST /complaints → CreateComplaint → DynamoDB → complaint ID returned to browser. Tracking flow: Browser → API Gateway GET /complaints/{complaintId} → TrackComplaint → DynamoDB → category and status returned. UpdateComplaint is a separate backend Lambda function.

5. Amazon S3 – frontend hosting

An S3 bucket named smart-complaint-frontend-2026 was created in us-east-1. Static website hosting was enabled with index.html as the index document. The frontend files index.html, style.css and script.js were uploaded. Public-read access was configured for the static website objects so the website endpoint could serve the files. The S3 website endpoint uses HTTP; API calls from JavaScript use HTTPS.

File	Purpose
index.html	Registration form and tracking interface
style.css	Layout, colours, buttons and responsive design
script.js	POST and GET fetch requests; response and error handling

S3 deployment steps: create the bucket → enable static website hosting → configure the index document → upload the three frontend files → configure public access for the website objects → open the S3 website endpoint and test registration and tracking. Do not upload credentials or private complaint records as public objects.

S3 Bucket – smart-complaint-frontend-2026

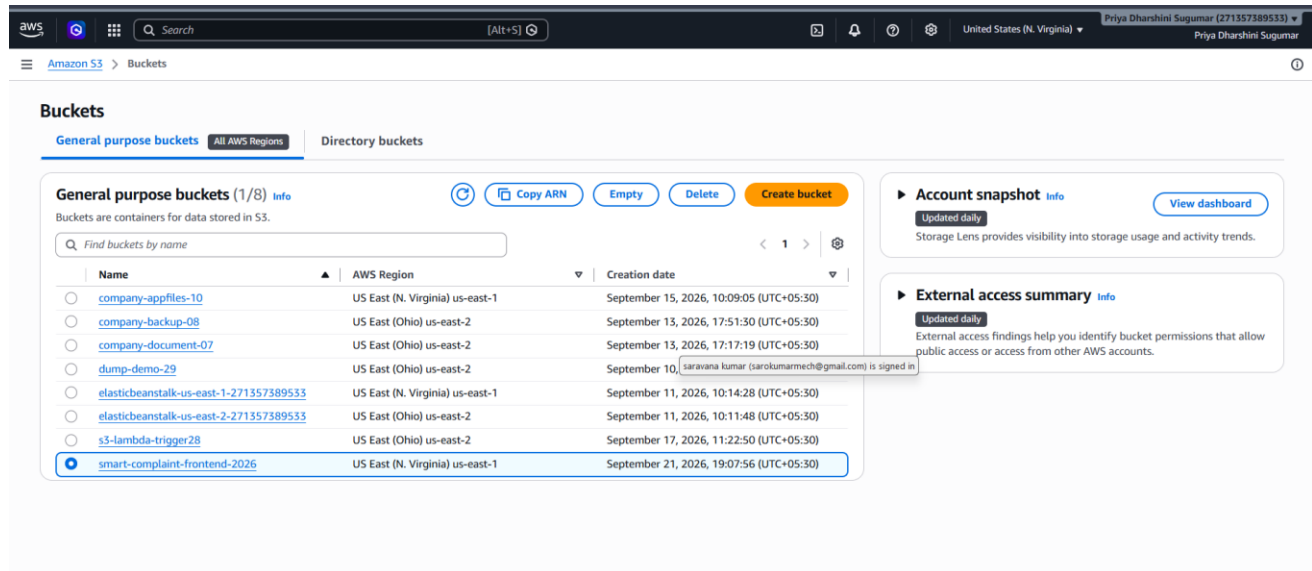


Figure 5.1: S3 bucket created for hosting the complaint management frontend.

Frontend Files Uploaded to Amazon S3

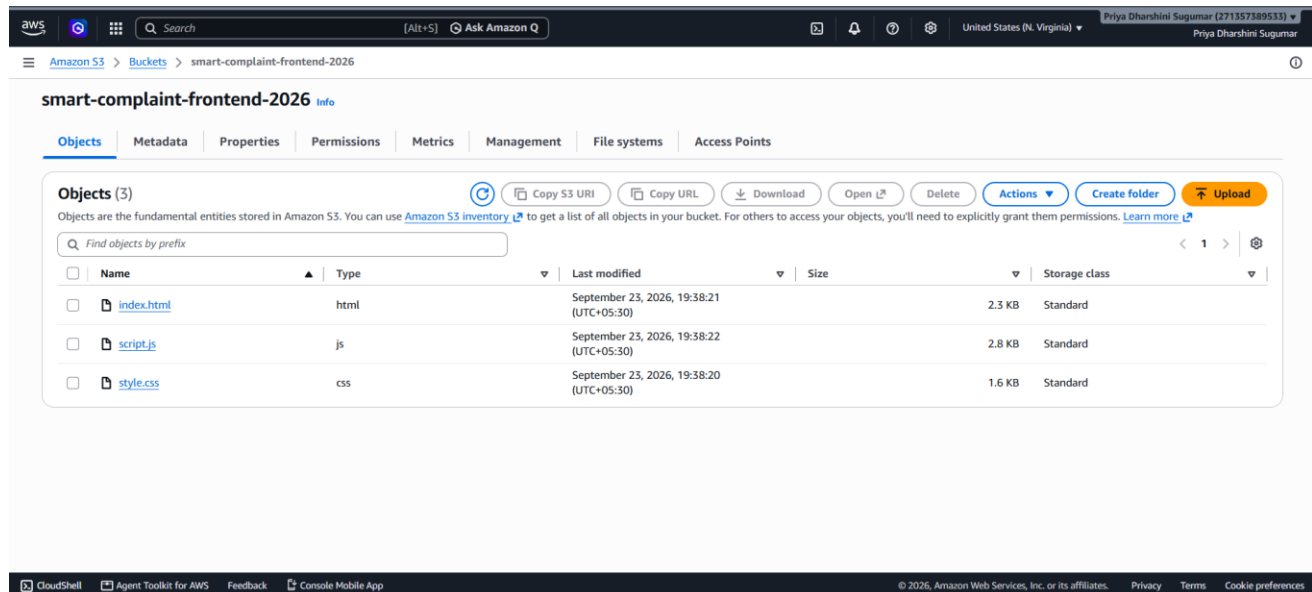


Figure 5.2: The S3 bucket contains index.html, style.css and script.js, which provide the web interface and API connectivity.

6. IAM – Lambda execution permissions

IAM execution roles allow each Lambda function to call DynamoDB without embedding AWS access keys in source code. CreateComplaint needs dynamodb:PutItem; TrackComplaint needs dynamodb:GetItem; UpdateComplaint needs dynamodb:UpdateItem on the Complaints table. Lambda also requires permissions to write its execution logs to CloudWatch Logs. Restrict resource permissions to the intended table where practical.

Lambda function	Required DynamoDB action	Purpose
CreateComplaint	dynamodb:PutItem	Insert a complaint record
TrackComplaint	dynamodb:GetItem	Read a complaint by complaintId
UpdateComplaint	dynamodb:UpdateItem	Change status of an existing complaint

Lambda Functions Created in AWS

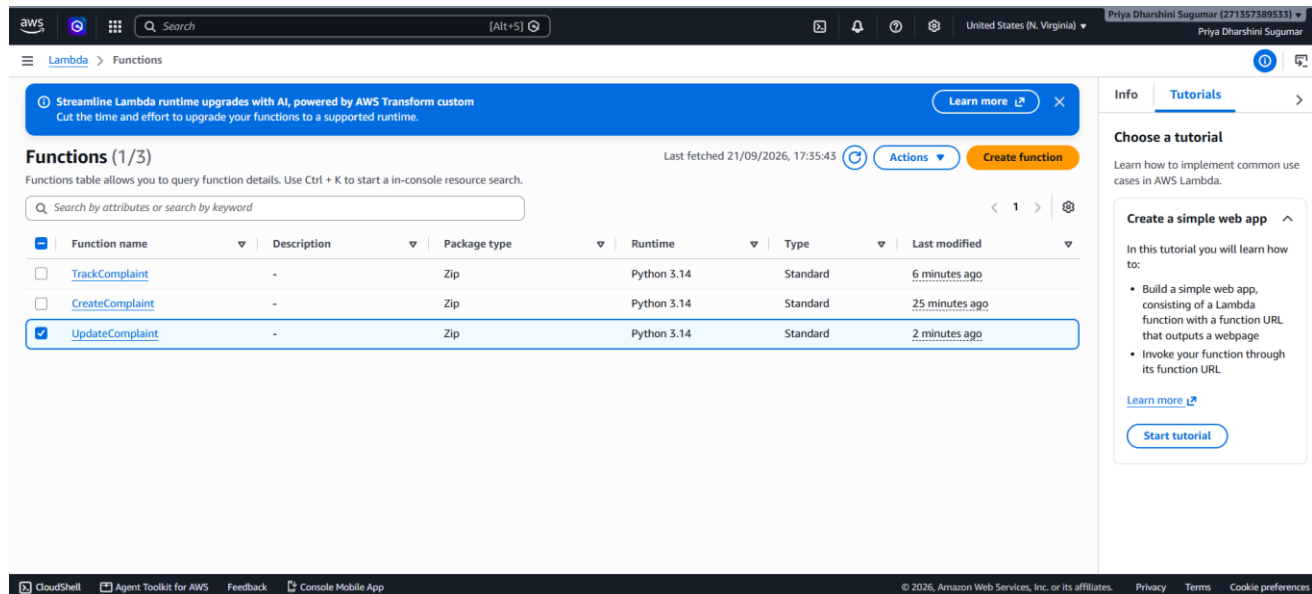


Figure 6.1: The AWS Lambda console lists the CreateComplaint, TrackComplaint and UpdateComplaint functions.

IAM setup workflow: Open IAM → create or select the Lambda execution role → attach the required DynamoDB permission for the Complaints table → assign the role to the relevant Lambda function → test the function. Each function uses an execution role to access AWS resources without storing access keys in application code.

IAM Role and DynamoDB Permissions – CreateComplaint Lambda

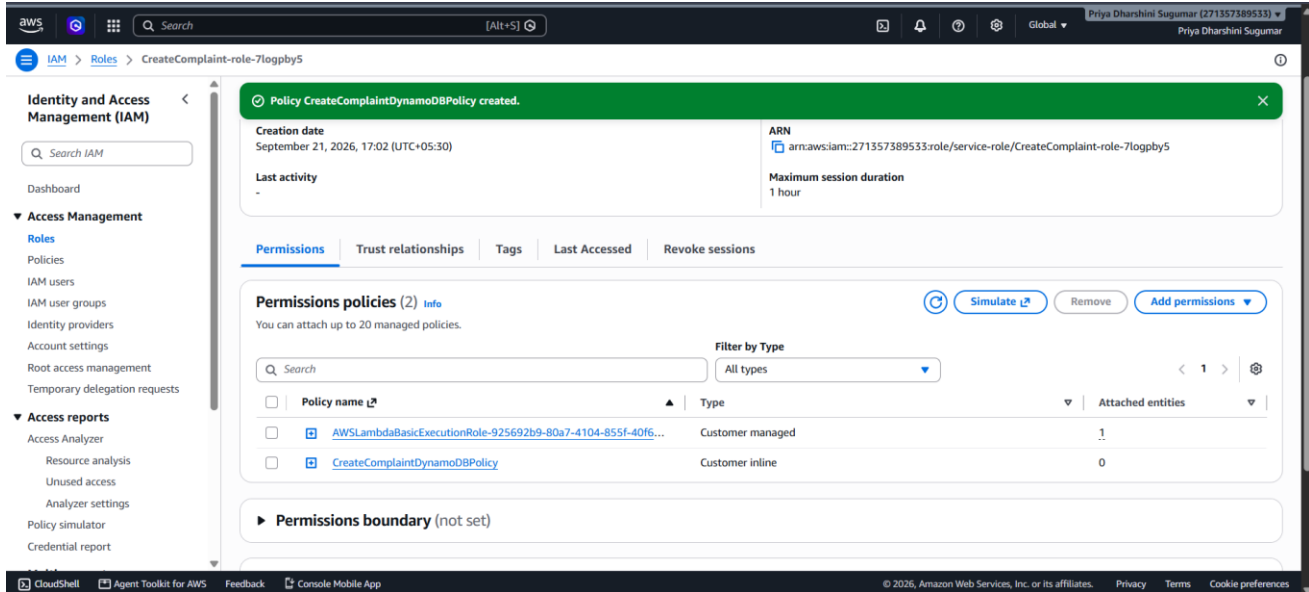


Figure 6.2: The CreateComplaint execution role has a DynamoDB permission policy attached for writing complaint records.

IAM Role and DynamoDB Permissions – TrackComplaint Lambda

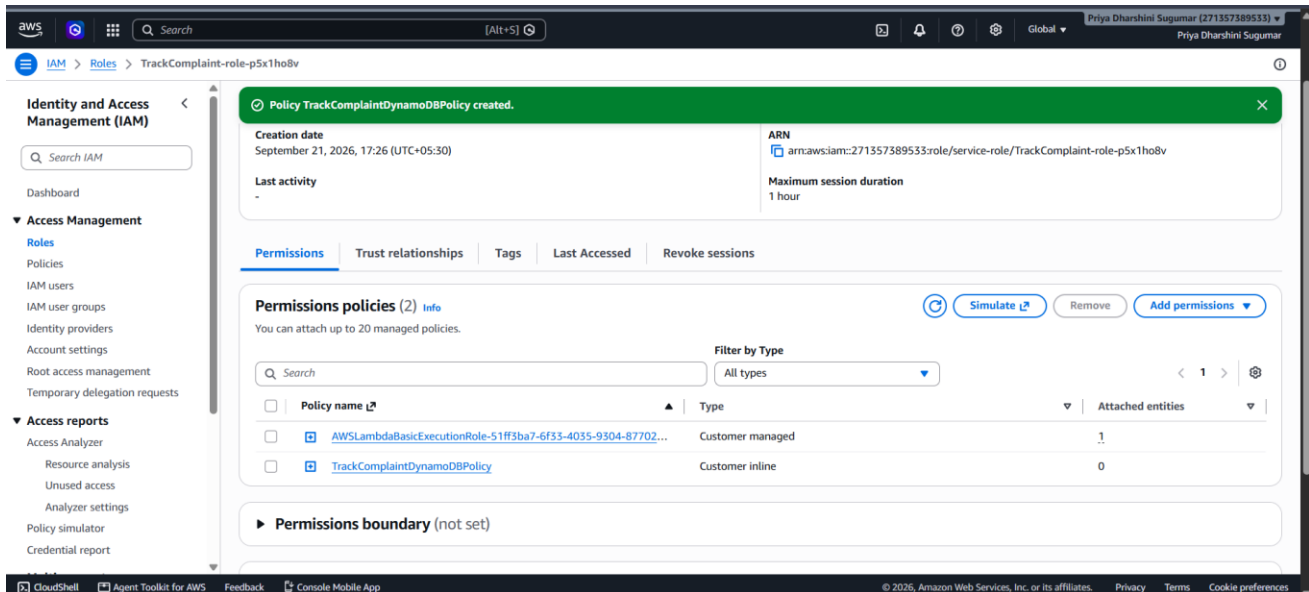


Figure 6.3: The TrackComplaint execution role has a DynamoDB permission policy attached for retrieving complaint records.

IAM Role and DynamoDB Permissions – UpdateComplaint Lambda

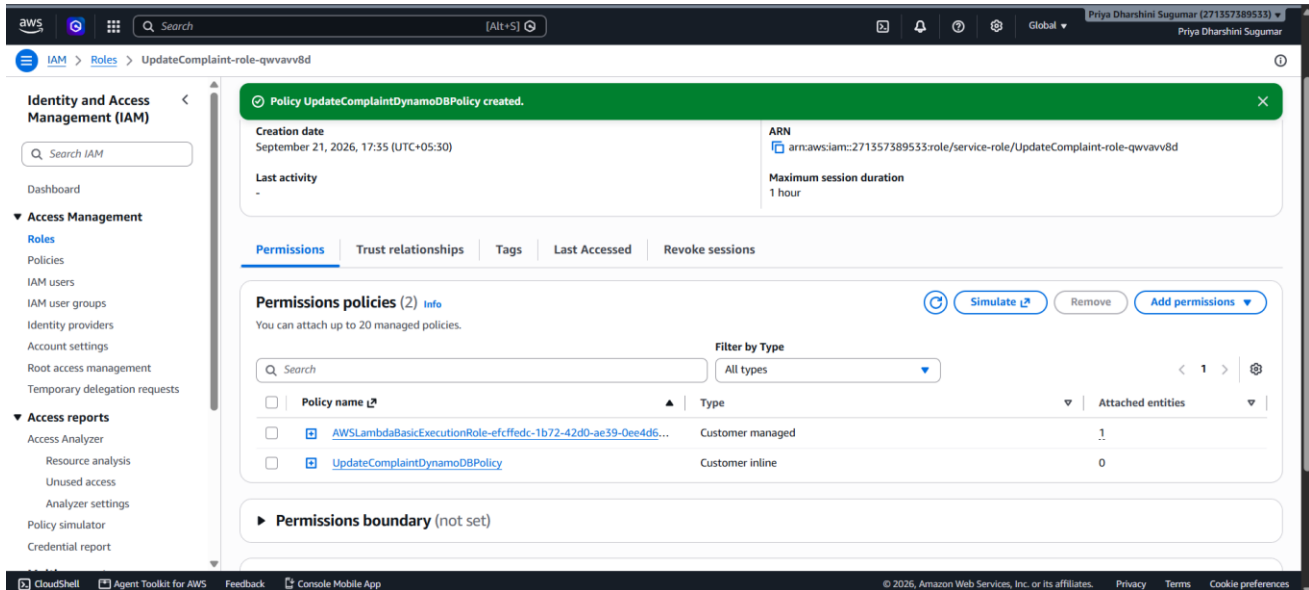


Figure 6.4: The UpdateComplaint execution role has a DynamoDB permission policy attached for updating complaint status.

7. DynamoDB – database design

The DynamoDB table is named Complaints. The partition key is complaintId. Each complaint item contains the submitted name, phone, category, location and description, together with its generated ID and status. CreateComplaint inserts an item; TrackComplaint retrieves an item using the primary key; UpdateComplaint changes the status when the item exists.

Attribute	Use
complaintId	Primary key / complaint reference
name	Submitter name
phone	Submitter phone
category	Complaint category
location	Complaint location
description	Complaint description
status	Pending initially; can be updated by backend

DynamoDB Complaints Table – Configuration

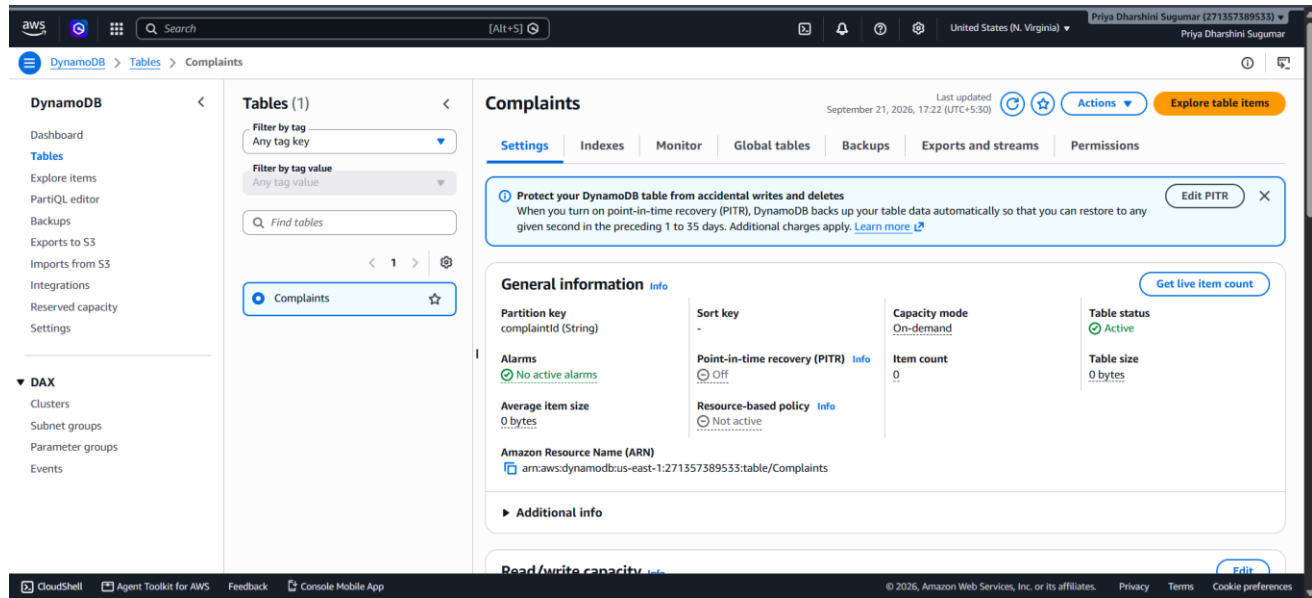


Figure 7.1: DynamoDB Complaints table configuration showing complaintId as the partition key.

8. AWS Lambda – implementation

The three Python functions use boto3 to access the Complaints table. API Gateway passes requests to the create and track functions through Lambda proxy integration.

8.1 CreateComplaint

Parses JSON request body, creates CMP- prefixed UUID-based ID, inserts the item and returns HTTP 201.

8.2 TrackComplaint

Reads complaintId from API Gateway pathParameters, retrieves the item and returns HTTP 200 or HTTP 404.

8.3 UpdateComplaint

Accepts complaintId and status, validates allowed status values and updates the existing item conditionally.

8.4 Lambda Console Testing – Sample Event JSON

The following sample JSON events illustrate the event structures for complaint registration, tracking and status updates in the AWS Lambda console.

8.4.1 CreateComplaint – Test Event

Purpose: To register a new complaint and store its details in Amazon DynamoDB.

Event JSON:

```
{
  "name": "Priya",
  "phone": "9876543210",
  "category": "Water",
  "description": "Water supply not available",
  "location": "Block A"
}
```

Expected Result: The function creates a complaint record with an initial status of Pending and returns HTTP 201 Created with a generated complaint ID.

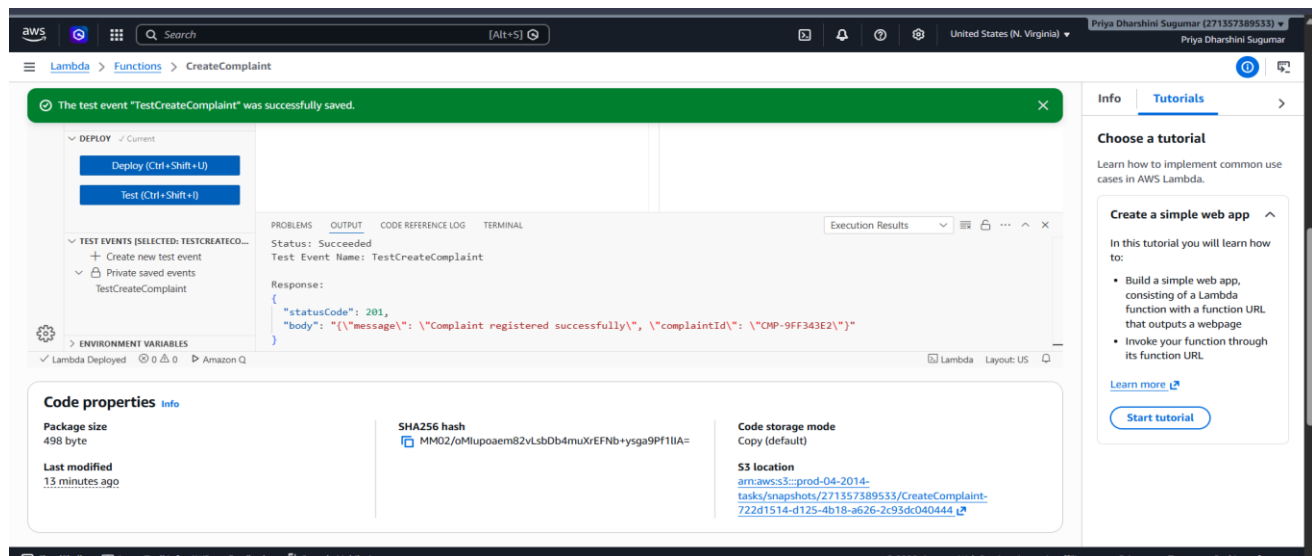
CreateComplaint – Lambda Test Result

Figure 8.1: The Lambda console displays the CreateComplaint test execution and its registration response.

8.4.2 TrackComplaint – Test Event

Purpose: To retrieve complaint details using an existing complaint ID.

Sample Event JSON:

```
{
  "pathParameters": {
    "complaintId": "CMP-F89E2C1B"
  }
}
```

Expected Result: The function returns HTTP 200 OK with the complaint ID, category, and status when the record exists. If the complaint is not found, it returns HTTP 404 Not Found.

TrackComplaint – Lambda Test Result

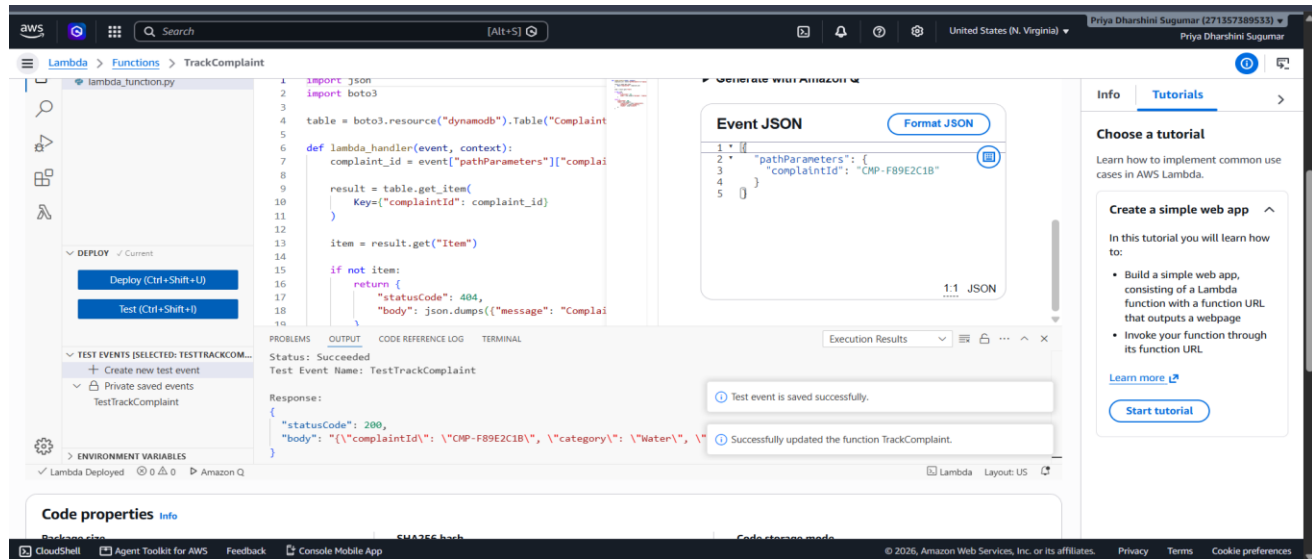


Figure 8.2: The Lambda console shows the `TrackComplaint` test event and the complaint details returned for the supplied ID.

8.4.3 UpdateComplaint – Test Event

Purpose: To update the status of an existing complaint in Amazon DynamoDB.

Sample Event JSON:

```
{
  "complaintId": "CMP-F89E2C1B",
  "status": "In Progress"
}
```

Expected Result: The function returns HTTP 200 OK for a successful update and HTTP 400 Bad Request for an invalid status.

UpdateComplaint – Lambda Test Result

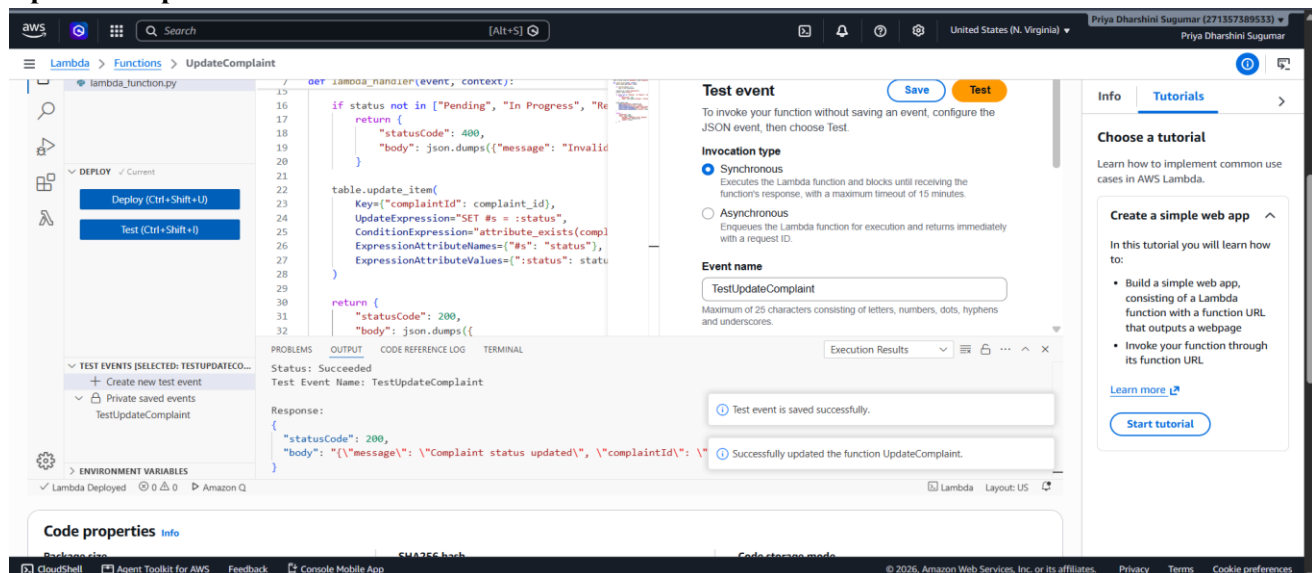


Figure 8.3: The Lambda console shows the `UpdateComplaint` test execution for a complaint status change.

9. API Gateway REST API and CORS

ComplaintManagementAPI is deployed to the dev stage. POST /complaints invokes CreateComplaint and GET /complaints/{complaintId} invokes TrackComplaint. Lambda proxy integration passes the request body and path parameters to the Python handlers. CORS response headers and OPTIONS handling support cross-origin browser requests from the S3-hosted frontend.

Method	Route	Backend	Expected outcome
POST	/complaints	CreateComplaint	HTTP 201 with complaintId
GET	/complaints/{complaintId}	TrackComplaint	HTTP 200 with ID/category/status or HTTP 404

The UpdateComplaint function is implemented separately; the current browser workflow and public REST API routes cover registration and tracking.

API Gateway REST API – POST Method Integration

The screenshot displays the AWS Management Console interface for the API Gateway REST API. A green notification banner at the top states: "Successfully created method 'POST' in 'complaints'. Redeploy your API for the update to take effect." The left sidebar shows the navigation menu with "API Gateway" selected. The main content area is titled "Resources" and shows the configuration for the "/complaints - POST - Method execution". The "Method request" tab is active, showing a flow diagram: Client → Method request → Integration request → Lambda integration. The "Integration request" is configured with a "Proxy integration" type. The "Method request settings" section is visible at the bottom.

API Gateway REST API – GET Method Integration

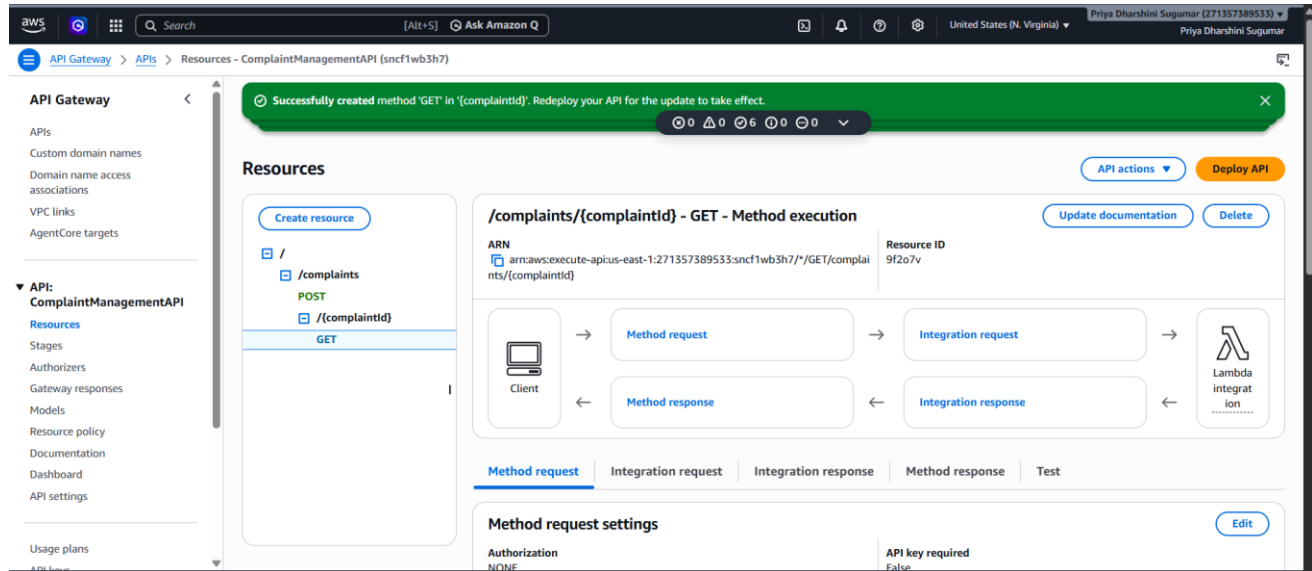


Figure 9.1: API Gateway GET /complaints/{complaintId} method is integrated with the TrackComplaint Lambda function.

9.1 API request and response examples

POST /complaints – illustrative JSON request:

```
{"name":"Test User","phone":"9876543210","category":"Maintenance","location":"Block 1","description":"Test issue"}
```

HTTP 201 – illustrative response:

```
{"message":"Complaint registered successfully","complaintId":"CMP-XXXXXXX"}
```

GET /complaints/{complaintId} – illustrative HTTP 200 response:

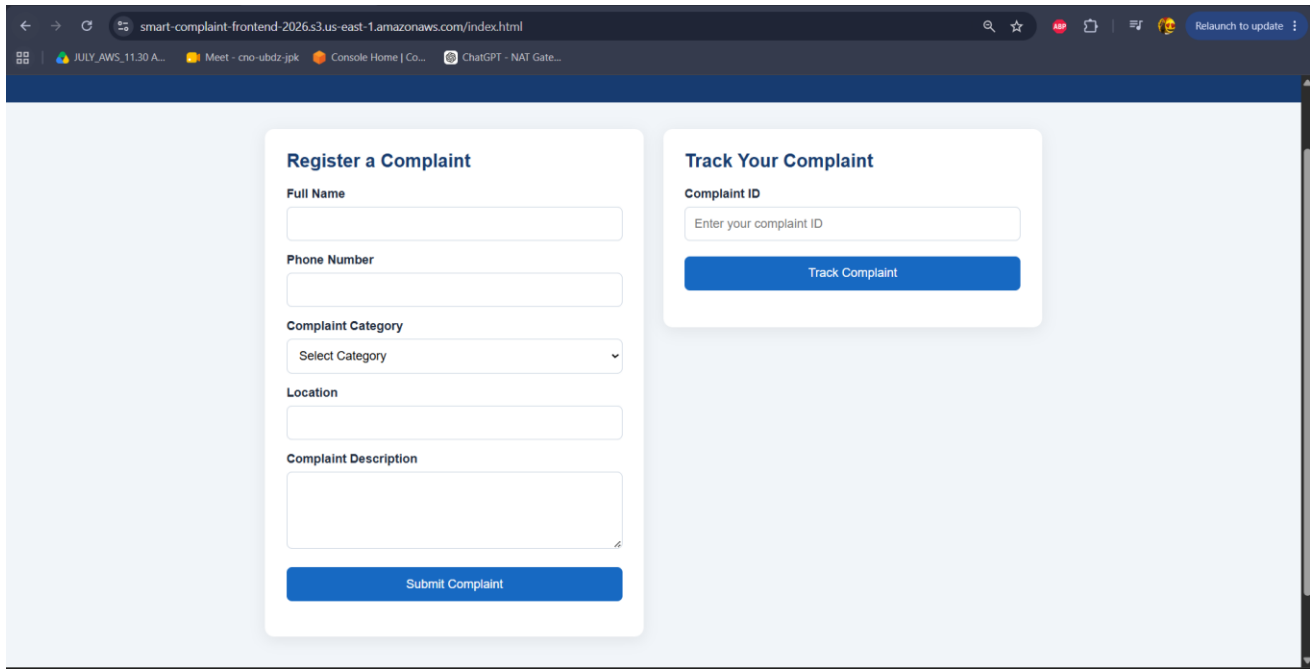
```
{"complaintId":"CMP-XXXXXXX","category":"Maintenance","status":"Pending"}
```

CMP-XXXXXXX is a placeholder, not a recorded complaint ID.

10. Deployment and testing

Test case	Expected result	Observed project outcome
Submit valid complaint	201 and generated complaint ID	Registration succeeded in browser testing
Track existing complaint	ID, category and status shown	Tracking succeeded in browser testing
Track unknown complaint	404 / not found message	Handled in TrackComplaint code
Invalid status update	400 / Invalid status	Handled in UpdateComplaint code

Complaint Registration Interface



The screenshot shows a web browser window with the URL `smart-complaint-frontend-2026.s3.us-east-1.amazonaws.com/index.html`. The interface consists of two main panels on a light blue background.

Register a Complaint Panel:

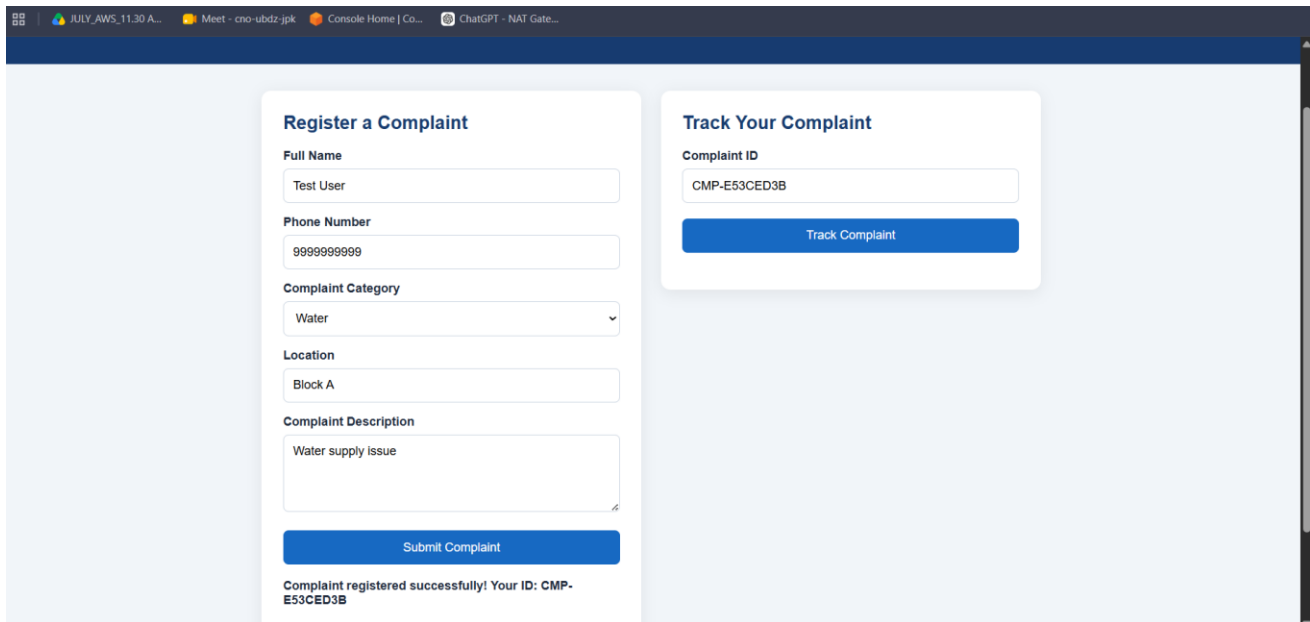
- Full Name:** A text input field.
- Phone Number:** A text input field.
- Complaint Category:** A dropdown menu with the option "Select Category".
- Location:** A text input field.
- Complaint Description:** A text area.
- Submit Complaint:** A blue button at the bottom.

Track Your Complaint Panel:

- Complaint ID:** A text input field with the placeholder "Enter your complaint ID".
- Track Complaint:** A blue button below the input field.

Figure 10.1: The S3-hosted website displays the complaint registration form and the complaint tracking panel.

Successful Complaint Registration



The screenshot shows the same web browser window as Figure 10.1, but with the registration form filled out and the tracking panel updated.

Register a Complaint Panel:

- Full Name:** "Test User"
- Phone Number:** "9999999999"
- Complaint Category:** "Water" (selected from the dropdown)
- Location:** "Block A"
- Complaint Description:** "Water supply issue"
- Submit Complaint:** A blue button at the bottom.

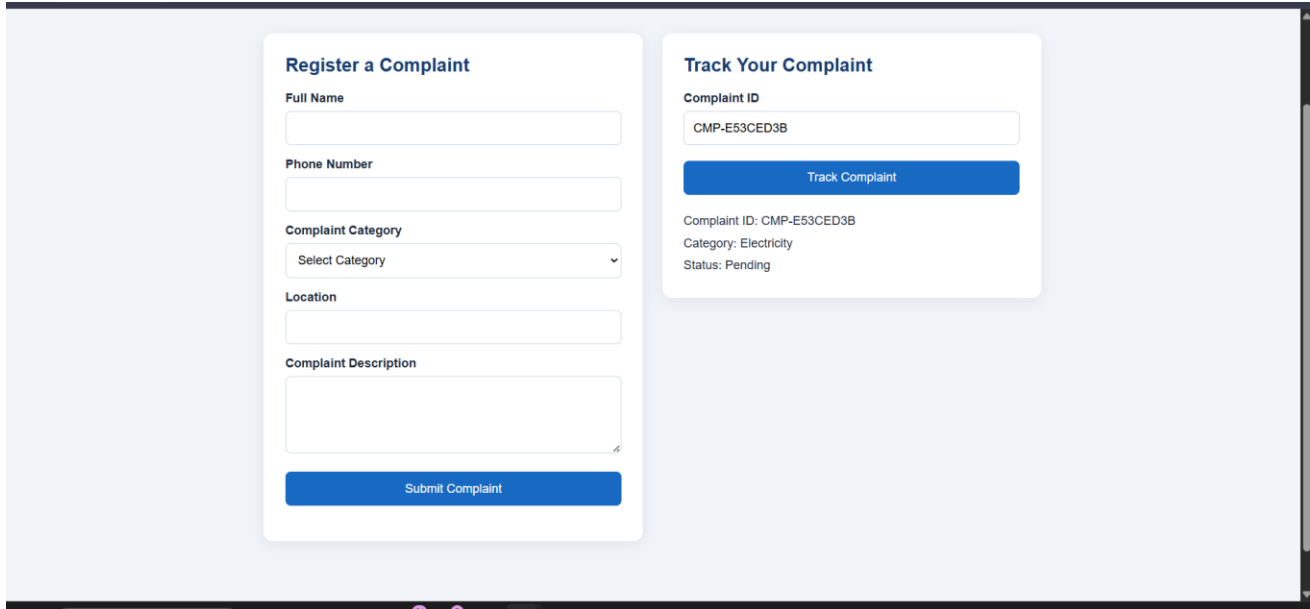
Track Your Complaint Panel:

- Complaint ID:** "CMP-E53CED3B" (generated and displayed in the input field)
- Track Complaint:** A blue button below the input field.

Below the "Submit Complaint" button, a message is displayed: "Complaint registered successfully! Your ID: CMP-E53CED3B".

Figure 10.2: After a valid complaint is submitted, the website displays a generated complaint ID for future tracking.

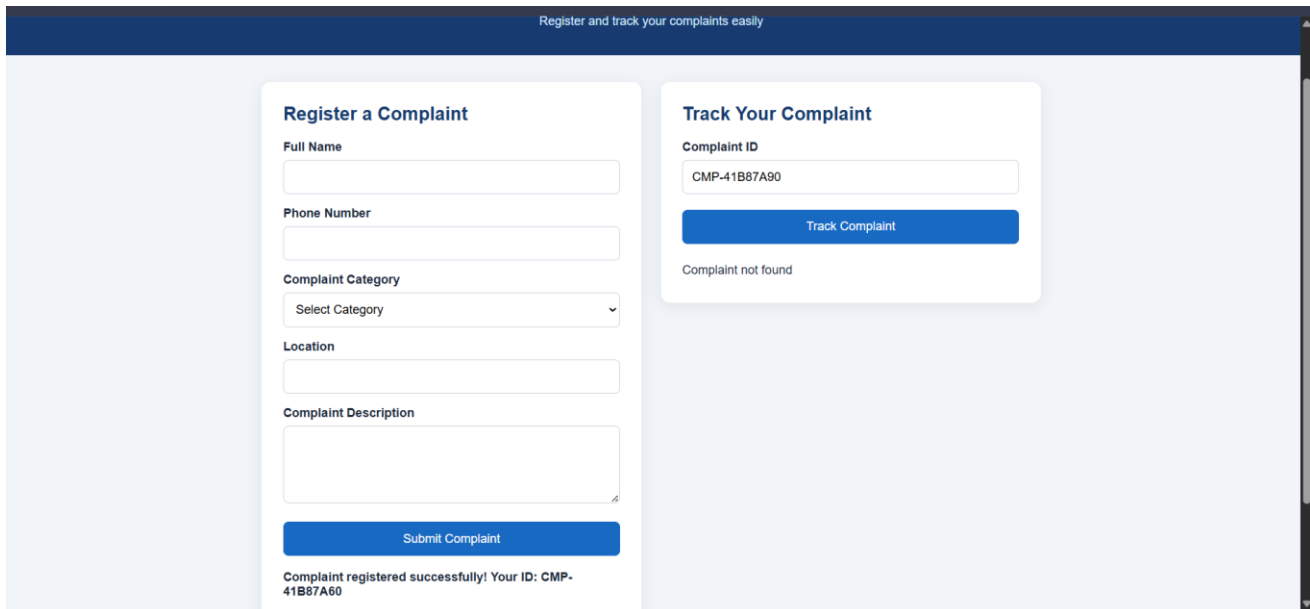
Successful Complaint Tracking



The screenshot displays two main panels on a light blue background. The left panel, titled 'Register a Complaint', contains input fields for 'Full Name', 'Phone Number', 'Complaint Category' (a dropdown menu), 'Location', and 'Complaint Description'. A blue 'Submit Complaint' button is at the bottom. The right panel, titled 'Track Your Complaint', shows a 'Complaint ID' input field with the value 'CMP-E53CED3B'. Below it is a blue 'Track Complaint' button. Underneath the button, the system displays the following information: 'Complaint ID: CMP-E53CED3B', 'Category: Electricity', and 'Status: Pending'.

Figure 10.3: The tracking panel retrieves and displays the complaint ID, category and current status.

Complaint Not Found – Tracking Result



The screenshot shows the same interface as Figure 10.3, but with a different complaint ID entered: 'CMP-41B87A90'. The 'Track Complaint' button is still present. Below the button, the system displays the message 'Complaint not found'. The 'Register a Complaint' panel on the left remains unchanged, but a success message is now visible at the bottom: 'Complaint registered successfully! Your ID: CMP-41B87A60'.

Figure 10.4: The website displays a complaint-not-found message when the entered ID does not match a stored record.

Complaint Records in DynamoDB

Table: Complaints - Items returned (7)
Scan started on September 21, 2026, 19:35:04

complaintId (String)	category	description	location	name	phone	status
CMP-F89E2C1B	Water	Water supply ...	Block A	Priya	9876543210	In Progress
CMP-00838A30	Water	Water supply ...	Block A	Priya	9876543210	Pending
CMP-E4A80FC3	Electricity	Street light n...	Block B	Priya	9876543210	Pending
CMP-08B1E4CC	Water	Water supply ...	Block A	Test User	999999999	Pending
CMP-D790F4C6	Water	Water supply ...	Block A	Priya	9876543210	Pending
CMP-9FF343E2	Water	Water supply ...	Block A	Priya	9876543210	Pending
CMP-E53CED3B	Electricity	electricity issue	block-a	priya	1564816878	Pending

Figure 10.5: The DynamoDB items view displays complaint records stored in the Complaints table.

11. Security and limitations

- The S3 website endpoint serves the static frontend over HTTP; an HTTPS-capable distribution can be added later.
- The create handler relies primarily on browser-side input validation; add server-side validation for a production system.
- UpdateComplaint is a separate backend function, not an administrator dashboard or publicly exposed update route.
- Temporary full-event debugging output should be removed from the deployed TrackComplaint function before processing real user data; the source listing in this report excludes that diagnostic statement.

12. Future enhancements and conclusion

Potential enhancements include authenticated administrator status updates, stricter API validation, access controls, HTTPS frontend delivery, monitoring and notification features. The implemented application demonstrates a serverless architecture combining Amazon S3, API Gateway REST API, AWS Lambda, IAM and DynamoDB for complaint registration and tracking.

Appendix A – Project source code

The following listings present the frontend files and the three Lambda functions used in this project. Temporary debugging statements are excluded from the TrackComplaint source listing.

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```

    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Smart Complaint Management System</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

<header>
    <h1>Smart Complaint Management System</h1>
    <p>Register and track your complaints easily</p>
</header>

<main>
    <section class="card">
        <h2>Register a Complaint</h2>

        <form id="complaintForm">
            <label for="name">Full Name</label>
            <input type="text" id="name" maxlength="100" required>

            <label for="phone">Phone Number</label>
            <input type="tel" id="phone"
                pattern="[0-9]{10}" maxlength="10" required>

            <label for="category">Complaint Category</label>
            <select id="category" required>
                <option value="">Select Category</option>
                <option value="Water">Water</option>
                <option value="Electricity">Electricity</option>
                <option value="Cleaning">Cleaning</option>
                <option value="Maintenance">Maintenance</option>
                <option value="Other">Other</option>
            </select>

            <label for="location">Location</label>
            <input type="text" id="location" maxlength="100" required>

            <label for="description">Complaint Description</label>
            <textarea id="description" rows="4"
                maxlength="1000" required></textarea>

            <button type="submit" id="submitButton">
                Submit Complaint
            </button>
        </form>

        <p id="submitMessage" class="message"></p>
    </section>

    <section class="card">
        <h2>Track Your Complaint</h2>

        <label for="trackId">Complaint ID</label>
        <input type="text" id="trackId"
            placeholder="Enter your complaint ID">

        <button type="button" id="trackButton">
            Track Complaint

```

```

        </button>

        <div id="trackingResult" class="result"></div>
    </section>
</main>

<footer>
    <p>AWS Serverless Microservices Project</p>
</footer>

<script src="script.js"></script>
</body>
</html>

```

style.css

```

* {
    box-sizing: border-box;
}

body {
    margin: 0;
    font-family: Arial, sans-serif;
    background: #f1f5f9;
    color: #1e293b;
}

header {
    background: #173b70;
    color: white;
    text-align: center;
    padding: 35px 15px;
}

header h1 {
    margin: 0 0 10px;
}

header p {
    margin: 0;
    color: #dbeafe;
}

main {
    max-width: 1050px;
    margin: 35px auto;
    padding: 0 18px;
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 25px;
    align-items: start;
}

.card {
    background: white;
    border-radius: 12px;
    padding: 28px;
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.07);
}

```

```
}

.card h2 {
  margin-top: 0;
  color: #173b70;
}

label {
  display: block;
  margin: 16px 0 7px;
  font-weight: bold;
}

input,
select,
textarea {
  width: 100%;
  padding: 12px;
  border: 1px solid #cbd5e1;
  border-radius: 7px;
  font: inherit;
}

button {
  width: 100%;
  margin-top: 20px;
  padding: 13px;
  background: #1769c2;
  color: white;
  border: none;
  border-radius: 7px;
  font-size: 16px;
  cursor: pointer;
}

button:hover {
  background: #10549c;
}

button:disabled {
  opacity: 0.6;
  cursor: wait;
}

.message {
  margin-top: 18px;
  font-weight: bold;
  overflow-wrap: anywhere;
}

.result {
  margin-top: 20px;
  line-height: 1.8;
  white-space: pre-line;
  overflow-wrap: anywhere;
}

footer {
  text-align: center;
```

```
padding: 25px;
color: #64748b;
}

@media (max-width: 750px) {
  main {
    grid-template-columns: 1fr;
  }
}
```

script.js

```
const API_URL =
  "https://sncf1wb3h7.execute-api.us-east-1.amazonaws.com/dev";

const form = document.getElementById("complaintForm");
const submitMessage = document.getElementById("submitMessage");
const submitButton = document.getElementById("submitButton");

const trackButton = document.getElementById("trackButton");
const trackingResult = document.getElementById("trackingResult");

// Register a complaint
form.addEventListener("submit", async function (event) {
  event.preventDefault();

  const complaint = {
    name: document.getElementById("name").value.trim(),
    phone: document.getElementById("phone").value.trim(),
    category: document.getElementById("category").value,
    location: document.getElementById("location").value.trim(),
    description: document.getElementById("description").value.trim()
  };

  submitMessage.textContent = "Submitting complaint...";
  submitButton.disabled = true;

  try {
    const response = await fetch(`${API_URL}/complaints`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify(complaint)
    });

    const result = await response.json();

    if (!response.ok) {
      throw new Error(
        result.message || "Complaint submission failed"
      );
    }

    submitMessage.textContent =
      "Complaint registered successfully! Your ID: " +
```

```

        result.complaintId;

        document.getElementById("trackId").value =
            result.complaintId;

        form.reset();

    } catch (error) {
        submitMessage.textContent = error.message;
    } finally {
        submitButton.disabled = false;
    }
});

// Track a complaint
trackButton.addEventListener("click", async function () {
    const complaintId =
        document.getElementById("trackId").value.trim();

    if (!complaintId) {
        trackingResult.textContent =
            "Please enter your complaint ID.";
        return;
    }

    trackingResult.textContent = "Checking complaint...";
    trackButton.disabled = true;

    try {
        const response = await fetch(
            `${API_URL}/complaints/${encodeURIComponent(complaintId)}`
        );

        const result = await response.json();

        if (!response.ok) {
            throw new Error(
                result.message || "Complaint not found"
            );
        }

        trackingResult.textContent =
            "Complaint ID: " + result.complaintId + "\n" +
            "Category: " + result.category + "\n" +
            "Status: " + result.status;

    } catch (error) {
        trackingResult.textContent = error.message;
    } finally {
        trackButton.disabled = false;
    }
});

```

create_complaint.py

```
import json
```

```

import uuid
import boto3

table = boto3.resource("dynamodb").Table("Complaints")

def lambda_handler(event, context):
    data = event.get("body", event)

    if isinstance(data, str):
        data = json.loads(data)

    complaint_id = "CMP-" + uuid.uuid4().hex[:8].upper()

    table.put_item(Item={
        "complaintId": complaint_id,
        "name": data["name"],
        "phone": data["phone"],
        "category": data["category"],
        "description": data["description"],
        "location": data["location"],
        "status": "Pending"
    })

    return {
        "statusCode": 201,
        "headers": {
            "Access-Control-Allow-Origin": "*",
            "Access-Control-Allow-Headers": "Content-Type",
            "Access-Control-Allow-Methods": "POST,OPTIONS"
        },
        "body": json.dumps({
            "message": "Complaint registered successfully",
            "complaintId": complaint_id
        })
    }

```

track_complaint.py

```

import json
import boto3

table = boto3.resource("dynamodb").Table("Complaints")

def lambda_handler(event, context):
    complaint_id = event["pathParameters"]["complaintId"]

    result = table.get_item(
        Key={"complaintId": complaint_id}
    )

    item = result.get("Item")

    if not item:

```

```

    return {
        "statusCode": 404,
        "headers": {
            "Access-Control-Allow-Origin": "*",
            "Access-Control-Allow-Headers": "Content-Type",
            "Access-Control-Allow-Methods": "GET,OPTIONS"
        },
        "body": json.dumps({
            "message": "Complaint not found"
        })
    }

    return {
        "statusCode": 200,
        "headers": {
            "Access-Control-Allow-Origin": "*",
            "Access-Control-Allow-Headers": "Content-Type",
            "Access-Control-Allow-Methods": "GET,OPTIONS"
        },
        "body": json.dumps({
            "complaintId": item["complaintId"],
            "category": item["category"],
            "status": item["status"]
        })
    }
}

```

update_complaint.py

```

import json
import boto3

table = boto3.resource("dynamodb").Table("Complaints")

def lambda_handler(event, context):
    data = event.get("body", event)

    if isinstance(data, str):
        data = json.loads(data)

    complaint_id = data["complaintId"]
    status = data["status"]

    if status not in ["Pending", "In Progress", "Resolved"]:
        return {
            "statusCode": 400,
            "body": json.dumps({"message": "Invalid status"})
        }

    table.update_item(
        Key={"complaintId": complaint_id},
        UpdateExpression="SET #s = :status",
        ConditionExpression="attribute_exists(complaintId)",
        ExpressionAttributeNames={"#s": "status"},
        ExpressionAttributeValues={":status": status}
    )

    return {

```

```
"statusCode": 200,  
"body": json.dumps({  
    "message": "Complaint status updated",  
    "complaintId": complaint_id,  
    "status": status  
})  
}
```